

CS 421 Homework #6: Temporal Difference Learning

Specification: Your assignment is to create an agent that uses temporal difference learning to learn proper behavior in the ReAntics environment.

Your final agent must reliably complete a game with the Random agent in 3 minutes or less when playing on a lab computer. In addition, your agent must always be able to complete a game when playing against itself. Finally, your agent should reliably defeat the Random agent more than 60% of the time.

Hints and Advice:

1. Once your code is working and configured the way you like, you'll need time to train it and make sure that training is working. So, try to complete your coding and debugging well in advance of the due date.
2. The ReAntics state space is much too big to learn in a short period of time. So you'll need a way to consolidate it. Specifically, design and build an approach for putting the states into a finite number of categories. For example, consider two states that are identical except that the queen ant's position is slightly different. It's probably safe to treat these as the same state, yes? So, rather than having a table with an entry for every state, you can have an entry for every *category* (subset of similar states). You will decide what these categories are.

Careful implementation of this state categorization is critical to the success of your agent. If you have too few categories the agent won't be able to learn. If you have too many, the agent will not make enough return visits to benefit from what it is learning. My best guess is that a good target would be on the order of tens of thousands of states. (An agent would see only a small fraction of these in any given game.) You should be prepared to try multiple designs to see which works best. Log how often each category is visited and decide if you are satisfied with it.

3. You will need to use a list (or other data structure) to keep track of the utility to each state-category your agent has encountered. Don't add entries to the list until your agent visits them for the first time. This will save you a lot of time and space.
4. To avoid losing utility values on a crash and help you stop/restart learning, write methods to save/load the current state-category-utility list to/from a file. Use a filename that contains your UP userid. Your agent should expect the file to be in the current working directory so that it will work properly on other computers besides your own. (**Example:** `"/smithj27_weights.txt"`) Each time your agent completes a game, save your current state utilities to a file. When the agent runs its first game after startup, it should look to see if a file is there and, if so, initialize the state-category-utilities list with the values from the file. (**Caveat:** When testing your agent, don't forget to delete this file when you need to reset the agent.)

5. Decide on a reward function for your agent. A good default is to use +1 for a win, -1 for a loss and a tiny negative number (e.g., -0.01) for everything else.
6. Select a discount factor (γ) for your TD-Learning algorithm. I recommend you start with 0.9 and then try different values later.
7. Select a learning rate (α) for your TD-Learning algorithm. I recommend you start with 0.1 and then try different values later. If you want to get fancy you can set this to a large number (say 0.9999) that declines exponentially over time to something small. This can greatly speed up agent learning at the risk of over-fitting.
8. Select an explore-vs-exploit strategy and apply it.
9. It's recommended that you use eligibility traces to speed up the agent's learning.
10. Your agent should use the learned utilities to decide what actions to take (active learning). Specifically, select the action that leads to the state with the highest utility. You can do this in a number of ways:
 - a. The easiest way to do this is to just use the prediction method from previous assignments that determine what the state would look like after the agent takes a given move. Then use your category utility table to find out what action leads to the state with highest utility. *Break ties randomly.*
 - b. The above method does not take into account the opponent's move. As an alternative, you can use your revised state prediction method from the minimax assignment that guesses the opponent's move by using applying the agent's knowledge as if it were its own opponent.
 - c. Learn over Q-values (state-action pairs) as discussed in lecture. This will require much more training but may give a more satisfying result as you are not giving the agent any free knowledge about how the environment works.
11. Don't code and pray! Diligently test your agent's behavior. Then, once it seems to be learning try tuning it by making adjustments to your categorization function, reward function, α , γ , etc. Change one thing at a time and train the agent for a relatively short time to see if it learns faster/better.

Turning In Your Assignment: Follow these steps:

1. Turn in your agent's source code by following the steps defined in homework #1.
2. Turn in a file that contains the category-utility values your agent learned via the second Turn it in Here link that is provided. You may have to compress this file to get it under the 10MB limit on Moodle.

Grading Guidelines

These are the same as homework #1.